30

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Yatay Otomatik Ölçekleme (HPA - Horizontal Pod Autoscaler)

Horizontal Pod Autoscaling (HPA): Traffic-Driven Fleet Scaling

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

E-ticaret sitenizde çılgın Cuma indirimi başladı! Sabah 3 pod ile başlayan Spring Boot servisiniz, binlerce müşteri akın edince terlemeye başladı (CPU kullanımı %80'e fırladı). HPA uykusundan uyanır, anında pod sayısını 3'ten 10'a çıkarır! Gece yarısı kampanya bitip müşteriler uyuyunca, fazla pod'ları sessizce kapatıp tekrar 3'e düşürür. Cebinizi boşuna bulut faturası ödemekten kurtarır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Black Friday flash sale madness! Your coffee delivery backend starts sweating as CPU utilization spikes past 80%. The Horizontal Pod Autoscaler (HPA) wakes up and automatically multiplies your pod count from 3 to 10 in 60 seconds! When midnight strikes and traffic dies, HPA scales the fleet back down to 3, saving you thousands on cloud bills!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

HPA, Metrics Server'dan aldığı metrikleri (CPU, Memory veya özel Prometheus metrikleri) sürekli sorgular. Ölçekleme formülü basittir: `İstenen Pod Sayısı = ceil(Mevcut Pod Sayısı * (Mevcut Metrik Değeri / Hedef Metrik Değeri))`. HPA ani dalgalanmalarda sistemin sürekli açılıp kapanmasını (flapping/thrashing) önlemek için 'cooldown/stabilization windows' (varsayılan: küçülme için 5 dakika bekleme) uygular.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

HPA queries metrics from the Metrics Server or Custom Metrics APIs. Its scaling algorithm calculates: `desiredReplicas = ceil[currentReplicas * (currentMetricValue / desiredMetricValue)]`. To prevent flapping (rapid oscillations), HPA enforces stabilization cooldown windows (defaulting to 5 minutes for downscaling) ensuring stable, predictable autoscaling behavior.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# coffee-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: coffee-hpa
  namespace: coffee-shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: coffee-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # CPU %70'i geçerse yeni pod ekle!

# Yük testi yaparak HPA'yı canlı izle:
# kubectl get hpa -w
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

HPA'nın çalışabilmesi için Deployment manifestonuzda pod'lar için MUTLAKA `resources.requests.cpu` tanımlanmış olmalıdır! Request tanımlı değilse HPA CPU yüzdesini hesaplayamaz (`<unknown>/70%`) ve ölçekleme yapmaz.

🇬🇧 WATCH OUT

HPA requires `resources.requests` defined on container specs! Without requests, HPA cannot calculate percentage utilization and stays stuck in `<unknown>`.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- HPA trafik patlamalarında otomatik olarak yeni pod kopyaları açar.
- Sakin zamanlarda pod sayısını düşürerek bulut maliyetini minimize eder.
- Metrics Server kümede kurulu ve aktif olmalıdır.

- HPA automatically provisions extra replicas during unpredictable traffic spikes.
- Scales down automatically during idle periods to reduce operational infrastructure bills.
- Metrics Server must be deployed in the cluster to expose node and pod metrics.